P0301R1
Jens Maurer <Jens.Maurer@gmx.net>
Audience: Evolution Working Group
2016-03-21

P0301R1: Wording for Unified Call Syntax (revision 1)

Introduction

The previous revision of this paper, P0301R0, attempted to faithfully provide a suggestion for alternative wording to implement the semantics outlined in P0251R0 Unified Call Syntax Wording by Bjarne Stroustrup and Herb Sutter, based on the EWG consensus from Kona:

- For `f(x,y)`, see if `f(x,y)` is valid and if so do that call; otherwise try `x.f(y)`.

The phrase "is valid" was clarified to mean "at least one viable function was found". This approach failed to achieve consensus in the WG21 plenary in Jacksonville.

Deviating from P0251R0 and its companion and predecessor papers, I now propose a separate syntax for achieving unified call semantics, also allowing chaining:

- `.f(x,y)` produces a merged overload set of `f(x,y)` and `x.f(y)` and invokes the best viable function, preferring the member function in case of a tie.
- `.x.f(y)` is equivalent to `.f(x,y)`
- chaining is supported: `.x.f(y).g(z)` is equivalent to `.g(.f(x,y), z)`

Design

See

- N4474 Unified Call Syntax: x.f(y) and f(x,y) (Bjarne Stroupstrup and Herb Sutter)
- P0131R0 Unified call syntax concerns (Bjarne Stroustrup)
- P0079R0 Extension methods for C++ (Jonathan Coe, Roger Orr)

As far as I understand, the main motivation for a unified call approach was to have a single call syntax addressing both member and non-member functions in the context of a template. This becomes more important as concepts allow to easily probe for a certain function call to be valid, but users are presumed not to care whether the functionality is provided by a member function or a non-member function.

The use-case to adapt classes to requirements expected by third-party templates is not supported, unless the new syntax is used:

```
// library 1
template<class T>
int f(const T& x) {
  return x.weight();       // cannot call non-member weight(S)
  // return .x.weight();   // would call non-member weight(S)
}

// library 2
struct S {
  int get_weight() const;
};

// adapter
int weight(const S& s) {
  return s.get_weight();
}

// application
S s;
int value = f(s);  // x.weight() ought to invoke weight(S), but is ill-formed
```

Examples

```
    struct S {
      int f();
      static bool g();
```

```
  int h(int);
  int (*fp)();
};
int f(int);
int h(S, char *);
int h(S, int *);
int (*fp)(S);

S s;
int x1 = .f(1);    // ok, calls ::f(int)
int x2 = .f(s);    // ok, equivalent to s.f()
bool x4 = .g(s);   // ok, equivalent to s.g()
int x5 = .h(s, 0); // ok, equivalent to s.h(0) (better match)
int x6 = .s.h(0);  // ok, equivalent to s.h(0) (better match)
int x7 = .h(s, nullptr);  // error: ambiguous
int x8 = .fp(s);   // ok, prefer s.fp() (member over non-member)
```

Open Issues

The wording proposal below still has defects in at least the following areas:

- in a postfix-expression linked to a unified call introducer, a class member access (5.2.5) is allowed to be applied to a non-class (but requires a function call, then)
- the function call section 5.2.2 needs to be checked again for the conditions when overload resolution applies, and when the given function is invoked directly. For example, does .x.f(y) invoke .f(x,y) (with full overload resolution) if "f" is a function pointer? [Suggested answer: Yes.]
- 14.2p4 might need an update so that the `template` keyword is not required for unified calls
- 5.2.2p4 needs to specify the argument list for a unified function call (which might not be the argument list given by the syntax)

Wording

Change 3.4.5 [basic.lookup.classref] paragraph 1:

> In a class member access expression (5.2.5 [expr.mem]) **that is not linked to a unified call introducer (5.2 [expr.post])**, if the . or -> token is immediately followed by an identifier followed by a <, the identifier must be looked up to determine whether the < is the beginning of a template argument list (14.2) or a less-than operator. The identifier is first looked up in the class of the object expression. If the identifier is not found, it is then looked up in the context of the entire postfix-expression and shall name a class template. **In a class member access linked to a unified call introducer where the . or -> token is immediately followed by an *identifier* followed by a <, the *identifier* shall name a template. [Note: When a class member access is linked to a unified call introducer and appears as the *postfix-expression* of a function call, both member and non-member lookup is performed; see 13.3.1.1 [over.match.call]. -- end note]**

Change the grammar for *postfix-expression* in 5.2 [expr.post]:

> *postfix-expression:*
>
> **.*opt*** *primary-expression*
>
> *postfix-expression [expr-or-braced-init-list]*
>
> *postfix-expression (expression-list$_{opt}$)*
>
> *...*

Add a new paragraph after 5.2 [expr.post] paragraph 2:

> **A *postfix-expression* that is a *primary-expression* prefixed by . (dot) is called a *unified call introducer*. Its type, value category, and value are the same as those of the *primary-expression*. [Note: A unified call introducer modifies the semantics of member and non-member function calls (5.2.2 [expr.call], 13.3.1.1 [over.match.call]). -- end note] A *postfix-expression* E1 is *linked* to a *postfix-expression* E2 if**
>
> - **E2 is the initial part of E1 or**
> - **E1 is linked to a *postfix-expression* E and E is linked to E2.**
>
> **A *full postfix-expression* is a *postfix-expression* such that no other *postfix-expression* is linked to it. A full *postfix-expression* that is linked to a unified call introducer shall either be a function call (5.2.2 [expr.call]) or be linked to one. [Example:**
>
> ```
> struct S {
>   struct M { int i; } m;
>   M f();
> ```

```
} s;
int x1 = s.m.i;       // ok
int x2 = .s.m.i;      // error: no function call
int x3 = .s.f().m.i;  // ok
```

-- end example]

Split and change 5.2.2 [expr.call] paragraph 1 - 3:

A function call is a ~~postfix expression~~ ***postfix-expression*** followed by parentheses containing a possibly empty, comma-separated list of *initializer-clause*s which constitute the arguments to the function.

If the *postfix-expression* is a pointer-to-member expression (5.5 [expr.mptr.oper]) of function type, that member function is selected. Otherwise, if the *postfix-expression* has class type, is an *id-expression*, optionally prefixed by . (dot), or is a class member access (5.2.5 [expr.mem] linked to a unified call introducer (5.2 [expr.post]), overload resolution is applied to select the best viable function (13.3.1.1 [over.match.call], 13.3 [over.match]). Otherwise, the *postfix-expression* shall be a prvalue of function pointer type or an lvalue of function type and the function pointed to or referred to is selected. [Note: The function-to-pointer standard conversion (4.3) is not applied. -- end note] ~~The postfix expression shall have function type or function pointer type. For a call to a non-member function or to a static member function, the postfix expression shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (4.3) is suppressed on the postfix expression), or it shall have function pointer type. Calling a function through an expression whose function type has a language linkage that is different from the language linkage of the function type of the called function's definition is undefined (7.5). For a call to a non-static member function, the postfix expression shall be an implicit (9.3.1, 9.4) or explicit class member access (5.2.5) whose id-expression is a function member name, or a pointer-to-member expression (5.5) selecting a function member; the call is as a member of the class object referred to by the object expression. In the case of an implicit class member access, the implied object is the one pointed to by this. [Note: a member function call of the form f() is interpreted as (*this).f() (see 9.3.1). -- end note] If a function or member function name is used, the name can be overloaded (Clause 13), in which case the appropriate function shall be selected according to the rules in 13.3 [over.match].~~

If the selected function is non-virtual, or if the *id-expression* in the class member access expression is a *qualified-id*, that function is called. Otherwise, its final overrider (10.3) in the dynamic type of the object expression is called; such a call is referred to as a virtual function call. [Note: the dynamic type is the type of the object referred to by the current value of the object expression. 12.7 describes the behavior of virtual function calls when the object expression refers to an object under construction or destruction. -- end note]

[Note: If a function or member function name is used, and name lookup (3.4) does not find a declaration of that name, the program is ill-formed. No function is implicitly declared by such a call. -- end note]

If the *postfix-expression* designates a destructor (12.4), the type of the function call expression is void; otherwise, the type of the function call expression is the return type of the statically chosen function (i.e., ignoring the virtual keyword), even if the type of the function actually called is different. This return type shall be an object type, a reference type or cv void.

Calling a function through an expression whose function type has a language linkage that is different from the language linkage of the function type of the called function's definition is undefined (7.5).

Change in 5.2.5 [expr.ref] paragraph 2:

For the first option (dot) the first expression shall have complete class type. For the second option (arrow) the first expression shall have pointer to complete class type. The expression E1->E2 is converted to the equivalent form (*(E1)).E2; the remainder of 5.2.5 will address only the first option (dot). [Footnote: ...] ~~In either case,~~ **Unless the class member access is linked to a unified call introducer (5.2 [expr.post]),** the *id-expression* shall name a member of the class or of one of its base classes. [Note: because the name of a class is inserted in its class scope (Clause 9), the name of a class is also considered a nested member of that class. -- end note] [Note: 3.4.5 describes how names are looked up after the . and -> operators. -- end note]

Add a new bullet in 13.3.3 [over.match.best] paragraph 1 after bullet 7:

- F1 and F2 are function template specializations, and the function template for F1 is more specialized than the template for F2 according to the partial ordering rules described in 14.5.6.2**, or, if not that,**
- **the context is a function call (13.3.1.1 [over.match.call]) linked to a unified call introducer and F1 is a member function and F2 is a non-member function**.

Change in 13.3.1.1 [over.match.call] :

In a function call (5.2.2 [expr.call])

postfix-expression (expression-list$_{opt}$ *)*

~~if the *postfix-expression* denotes a set of overloaded functions and/or function templates, overload resolution is applied as specified in 13.3.1.1.1 [over.call.func]. If the *postfix-expression* denotes an object of class type, overload resolution is applied as specified in 13.3.1.1.2 [over.call.object].~~ **the set of candidate functions is determined from the *postfix-expression* and the argument types in the *expression-list*.**

- **If the *postfix-expression* is a unified call introducer (5.2 [expr.post]) or is linked to one, the set of candidate functions is determined as specified in 13.3.1.1.3 [over.call.unified].**
- **Otherwise, if the *postfix-expression* has class type, the set of candidate functions is determined as specified in 13.3.1.1.2 [over.call.object].**
- **Otherwise, the set of candidate functions is determined as specified in 13.3.1.1.1 [over.call.func].**

Change in section 13.3.1.1.2 [over.call.object] paragraph 1:

If the ~~*primary-expression*~~ ***postfix-expression*** E in the function call syntax evaluates to a class object of type "cv T", then the set of candidate functions includes at least the function call operators of T. The function call operators of T are obtained by ordinary lookup of the name operator() in the context of (E).operator().

Add a new section 13.3.1.1.3 [over.call.unified]:

13.3.1.1.3 Unified function call [over.call.unified]

A partial candidate set is determined from a result of name lookup as follows:

- **For a variable of type pointer to function (or reference thereto), the set consisting of a surrogate call function constructed from its function type;**
- **otherwise, for an object of class type, the candidate set determined for a call to an object of class type (13.3.1.1.2 [over.call.object]);**
- **otherwise, the candidate set determined for a call to a named function (13.3.1.1.1 [over.call.func]).**

If the *postfix-expression* in the function call syntax is a unified call introducer (5.2 [expr.post]), the set of candidate functions is the union of the following sets:

- **If name lookup (3.4 [basic.lookup]) for the *primary-expression* finds one or more class members, the set of functions found by argument-dependent lookup (3.4.2 [basic.lookup.argdep]),**
- **the partial candidate set from the result of name lookup, and**
- **if the first element of the *expression-list* (if any) has class type, the partial candidate set from a class member access lookup (3.4.5 [basic.lookup.classref]) where the first element of the *expression-list* is the object expression and the *id-expression* is the *primary-expression* of the function call, in which case the argument list consists of the remainder of the *expression-list*.**

[Example:

```
namespace N {
  struct A { };
  void f();
}
int (*f)();
struct S {
  int f(N::A);
  int x = .f(S(), N::A());  // all functions and function pointers named "f" are candidates
                            // only S::f(N::A) is viable
};
```

-- end example]

If the *postfix-expression* is a class member access linked to a unified call introducer, the set of candidate functions is the union of the following sets:

- **The partial candidate set from the result of class member access lookup (3.4.5 [basic.lookup.classref]) and**
- **the partial candidate set from a lookup of the *primary-expression* and the set of functions found by argument-dependent lookup; in both cases the object expression is prepended to the *expression-list*.**

[Example:

```
namespace N {
  struct A { };
```

```
  void f();
}
int (*f)();
struct S {
  int f(N::A);
  int x = .S().f(N::A());  // all functions and function pointers named "f" are candidates;
                           // only S::f(N::A) is viable
};
```

-- end example]

[Example:

```
struct S {
  int f();
  static bool g();
  int h(int);
  int (*fp)();
};
int f(int);
int h(S, char *);
int h(S, int *);
int (*fp)(S);

S s;
int x1 = .f(1);    // ok, calls ::f(int)
int x2 = .f(s);    // ok, equivalent to s.f()
bool x4 = .g(s);   // ok, equivalent to s.g()
int x5 = .h(s, 0); // ok, equivalent to s.h(0) (better match)
int x6 = .s.h(0);  // ok, equivalent to s.h(0) (better match)
int x7 = .h(s, nullptr);  // error: ambiguous
int x8 = .fp(s);   // ok, prefer s.fp() (member over non-member)

struct X {
  S m;
} x;
int x9 = .f(g(s));           // error; S::g is not a candidate
int x9b = .f(.g(s));         // ok; equivalent to ::f(s.g())
int q(int, int);
int x10 = .x.m.f().q(5);     // ok; equivalent to .q(.f(x.m), 5)
int x11 = .(x.m.f()).q(7);   // ok; equivalent to .q(x.m.f(), 7)
int x12 = (.x.m.f()).q(8);   // error: no member "q" in non-class type "int"
```

-- end example]

Change in 14.6.2 [temp.dep] paragraph 1:

... In an expression of the form:

postfix-expression (expression-list$_{opt}$ *)*

where the *postfix-expression* is an *unqualified-id* **optionally prefixed by . (dot) or a class member access (5.2.5 [class.mem]) linked to a unified call introducer (5.2 [expr.post]) with the *id-expression* of the class member access an *unqualified-id***, the *unqualified-id* denotes a dependent name if

- any of the expressions in the expression-list is a pack expansion (14.5.3)
- any of the expressions or *braced-init-list*s in the *expression-list* is type-dependent (14.6.2.2), ~~or~~
- **in the case of a class member access, the object expression is type-dependent, or**
- the *unqualified-id* is a *template-id* in which any of the template arguments depends on a template parameter.

Acknowledgements

The `.f(x)` syntax or slight variations thereof were proposed by several parties informally, including Mikhail Semenov in c++std-ext-16355, Dawn Perchik, and Daveed Vandevoorde. Thanks to Dawn Perchik for suggestions on the wording.